IKANO

Senior / Staff / Architect

Java Engineering Role Brief

Modern banking services, AWS architecture, practical engineering leadership, and systems that teams can own and improve.

01

Java services

02

AWS architecture

03

Technical leadership

What this role is about

Build useful things, simplify complexity, improve engineering quality, mentor others, and help shape systems that run reliably in a regulated banking environment.

IKANO

Role description

Ikano Bank is building a modern bank that ships faster and runs better. This senior-focused brief is for engineers who can build, simplify, mentor, and make sound technical tradeoffs.

Role setup

Permanent, full-time. Offices: Stockholm, Wiesbaden, Warsaw, Nottingham, Helsinki and Oslo. For senior-level and above, remote within the EU is also open.

1 Build & improve

Java services
Applications and tools

- Design, build, test and maintain Java services.
- Work with APIs, integrations, jobs and data flows.
- Contribute to customer journeys and internal tools.

2 Shape systems

Architecture
Reliability and cloud

- Improve maintainability, observability and reliability.
- Make practical architecture decisions.
- Contribute to AWS and platform design choices.

3 Raise the bar

Senior impact
Teams and standards

- Mentor engineers and support code reviews.
- Create clarity in ambiguous situations.
- Drive quality, ownership and better ways of working.

Technical space

Core engineering

- Java, Spring Boot, backend development and APIs.
- Relational databases, SQL, JPA and data modelling.
- JUnit, Git and collaborative delivery.

Web and product flow

- Web application fundamentals.
- Thymeleaf or Freemarker templates.
- HTMX interest is a strong plus.

Cloud and architecture

- Strong AWS knowledge for senior candidates.
- Cloud-native tradeoffs, platform decisions, security.
- Microservices, events or distributed systems are a plus.

Regulated context

- Banking, fintech or regulated environments are useful.
- Risk, auditability and operational discipline matter.
- Practical decisions beat buzzwords.

IKANO

Senior expectations and interview focus

For senior, staff and architect candidates, the strongest signal is judgment: the ability to balance speed, simplicity, risk and maintainability while helping teams improve over time.

What good looks like

1. **Own the problem**
   Understand the product, customer, operational and regulatory context before proposing a solution.
2. **Design for change**
   Prefer simple, understandable systems that can evolve without trapping the team.
3. **Build for production**
   Think about observability, reliability, failure modes, data, security and support from the start.
4. **Lead through clarity**
   Explain tradeoffs plainly, mentor others, and turn ambiguity into workable direction.
5. **Raise standards**
   Improve code, reviews, documentation, delivery habits and ownership across teams.

Interview and hiring signals

Architecture judgment
Clear tradeoffs, not diagram theatre.

Code quality
Readable design and sensible responsibilities.

Production mindset
Reliability, observability, security and support.

Cloud decisions
Practical AWS choices; knows cost and risk.

Team leadership
Mentors, unblocks and improves ways of working.

Communication
Explains complex topics simply and honestly.

Why join

This is for people energised by real products, useful software, straightforward colleagues, shipping things that matter, and simplifying complexity rather than adding to it.

Benefits include pension, wellness allowance, flexible working, annual social day, employee banking benefits, and learning support.

Candidate take-home task

Build a sample onboarding flow web app

The assignment is to build a small web application that demonstrates how an Ikano-style customer onboarding flow could work across countries, customer types, and external checks.

1. First choice

- User selects country: Sweden, Spain, or Poland.
- Then selects account type: private individual or business.

2. Adaptive journey

- Render different steps based on country and account type.
- Show progress and validation.

3. Mock integrations

- Mock KYC, KYB, sanctions, credit bureau, and registry checks.
- Handle pass, manual review, and fail outcomes.

Scenario

A customer lands in the app and starts an onboarding application. The application may be for a private customer or a business customer. The product team wants to evaluate how easily the flow can be adapted per market, how safely data is handled, and how external decisioning can be integrated without making the codebase brittle.

We are not asking for a real banking integration. We are asking for a practical, well-structured sample that shows product judgement, Java engineering, data modelling, and production thinking.

Timebox and scope

- Recommended timebox: 6-8 hours. Prioritize clarity, structure, and reasoning over visual polish.
- Use a supported LTS Java release. Spring Boot or another sensible Java stack is fine; use Maven or Gradle.
- A server-rendered web app with Thymeleaf or Freemarker is acceptable; HTMX or small JavaScript enhancements are welcome but not required.
- Use H2 or PostgreSQL. Persist the onboarding application, selected flow, step status, integration results, and final decision.
- Document any assumptions. We care more about explicit tradeoffs than legal perfection.

Core requirements

- Country and account-type selector leading to one of six flows.
- At least one form per step with meaningful validation and user feedback.
- Mocked external integrations with deterministic responses and failure cases.
- Decision outcome: approved, referred to manual review, or rejected.
- Basic audit trail showing what was checked, when, and with what result.
- Readable README, runnable locally with Maven or Gradle, with tests for the important logic.

Minimum flow matrix

Private individual onboarding

These are suggested minimum steps. Candidates may improve the flow if they document why. The integrations must be mocked.

Country	Suggested steps	Mock integrations	Decision data
Sweden	1. Choose Sweden + private individual 2. Collect personal identity number and initiate BankID-style identity check 3. Confirm contact details and address 4. Capture consent, PEP/sanctions declaration, and tax residency 5. Collect employment, income, household and affordability inputs 6. Run credit-bureau decision and affordability rules 7. Review summary, accept terms, submit application	Identity/KYC mock: BankID-style success/fail/manual review Address lookup mock PEP/sanctions mock Credit bureau + affordability mock	Identity confidence, income, debt flags, affordability result, final decision
Spain	1. Choose Spain + private individual 2. Collect DNI/NIE and initiate Clave/DNIe/document-verification mock 3. Confirm contact details, province, address, and tax residency 4. Capture consent and PEP/sanctions declaration 5. Collect employment, income, housing costs and dependants 6. Run credit-bureau and affordability decision 7. Review summary, accept terms, submit application	Identity mock: DNI/NIE + Clave/DNIe or document check Address/province validation mock PEP/sanctions mock Credit bureau + affordability mock	DNI/NIE validity, address confidence, income, bureau score, manual review reasons
Poland	1. Choose Poland + private individual 2. Collect PESEL and initiate eID/Trusted Profile/mObywatel-style identity mock 3. Confirm contact details and registered address 4. Capture consent and PEP/sanctions declaration 5. Collect employment, income and affordability information 6. Run BIK-style credit-bureau mock and decision rules 7. Review summary, accept terms, submit application	Identity mock: PESEL + eID-style verification Address validation mock PEP/sanctions mock Credit bureau + affordability mock	PESEL validity, identity result, employment/income, bureau score, final decision

Implementation note

The exact country rules are not the point of the exercise. The point is to make the flow adaptable, explicit and testable. Good solutions separate flow configuration, validation, integration calls, and decisioning logic.

Minimum flow matrix

Business onboarding

Business onboarding should feel different from individual onboarding: legal entity verification, representatives, beneficial owners, business risk and authority to act matter.

Country	Suggested steps	Mock integrations	Decision data
Sweden	1. Choose Sweden + business 2. Collect organisation number, legal name and legal form 3. Run Bolagsverket-style company registry lookup 4. Confirm authorised representative and signatory rights 5. Collect beneficial owners and verify them with BankID-style KYC mock 6. Capture business activity, turnover, purpose and expected usage 7. Run KYB, sanctions/PEP and business-credit decision, then review/sign	Company registry mock: organisation number, status, directors Representative/signatory mock UBO KYC + sanctions mock Business credit/risk mock	Company status, signatory authority, UBO risk, business activity, final decision
Spain	1. Choose Spain + business 2. Collect company NIF, legal form and registered address 3. Run Registro Mercantil / tax-status-style lookup 4. Verify legal representative using DNI/NIE identity mock 5. Collect beneficial owners and ownership percentages 6. Capture sector, turnover, VAT/tax details and expected usage 7. Run KYB, sanctions/PEP, business credit and IBAN verification, then review/sign	Company registry mock: NIF, status, directors, filings Representative identity + authority mock UBO KYC + sanctions mock Business credit + bank account mock	NIF status, representative authority, ownership risk, sector risk, manual review reasons
Poland	1. Choose Poland + business 2. Collect NIP, REGON or KRS/CEIDG identifier and legal form 3. Run CEIDG/KRS-style registry lookup 4. Confirm board member or sole proprietor authority 5. Collect beneficial owners and verify identity/risk 6. Capture VAT/tax status, business activity and expected usage 7. Run KYB, sanctions/PEP, business credit and bank-account validation, then review/sign	Company registry mock: CEIDG/KRS, NIP, REGON, status Representative/board authority mock UBO KYC + sanctions mock Business credit + bank account mock	Registry status, authority, VAT/tax flags, UBO risk, company credit result

Good senior signal

A strong solution avoids a large nested if/else tree. It should be possible to add a new country, a new customer type, or a new verification step without rewriting the whole app.

Engineering expectations

Build expectations and mock services

Keep the app simple, but make it feel like the foundation of a real regulated product flow.

Recommended architecture

1. Web layer

- step pages, form validation, progress indicator, and review screen.

2. Flow engine

- country/type-specific step definitions and allowed transitions.

3. Application state

- selected flow, step completion, answers, integration results, status and timestamps.

4. Integration layer

- small mock clients with typed request/response objects.

5. Decisioning layer

- deterministic rules mapping integration results to approved, manual review, or rejected.

6. Audit layer

- log customer actions and integration outcomes without leaking sensitive details.

Mock integration requirements

Do not call real KYC, registry, credit-bureau, or bank services. Build deterministic mocks that behave like external services.

Mock service	Examples of outputs	What we want to see
Identity / KYC	verified, document_mismatch, expired_id, manual_review	Clear client boundary, validation, failure handling
KYB / registry	active_company, dissolved, unknown_representative, missing_ubo	Country-specific identifiers and business rules
PEP / sanctions	no_hit, possible_hit, confirmed_hit	Manual review path and audit record
Credit / affordability	score, debt_flags, disposable_income, decision_reason	Simple deterministic decisioning and test cases
Bank account	iban_verified, name_mismatch, unreachable	Simulated timeouts/retries or graceful fallback

Production mindset

- Validate input server-side and handle all integration outcomes explicitly.
- Avoid logging personal identifiers or sensitive answers in raw form.
- Use request IDs or event IDs so a support person can trace an application.
- Add JUnit tests around flow transitions, decisioning and mocked integration cases.

Delivery and assessment

Bonus: resumable onboarding

The bonus is to make the flow resumable when a customer drops out halfway through. This is a strong signal because it touches state management, security, product thinking and idempotency.

A good resumability design could include:

- Save progress after every completed step.
- Resume at the last incomplete step, not always at the start.
- Use a resume token or magic link with expiry rather than a raw application ID.
- Show clear progress and what is still missing.
- Avoid re-running expensive or sensitive checks unless the input changed.
- Handle abandoned, expired, submitted and manually reviewed applications distinctly.

Deliverables

- A Git repository or zip with the application code.
- README with Java version, Maven/Gradle setup, run instructions, demo data and assumptions.
- Short architecture note: key packages/modules, data model, flow configuration, and tradeoffs.
- JUnit tests for important flow logic and mock integration outcomes.
- Optional: screenshots, short loom-style walkthrough, or simple deployment notes.

What we do not expect

- No real national eID, KYC, company registry, credit bureau or banking API integrations.
- No production authentication system, cloud deployment, or pixel-perfect UI.
- No attempt to make legally definitive country compliance claims. Mock reasonable market-specific checks and document assumptions.

Evaluation rubric

Area	Weight	What strong looks like
Product flow and UX	15%	Clear flow, sensible steps, validation, progress and review screen.
Architecture and data model	20%	Simple boundaries, maintainable state model, adaptable flow definitions.
Mocks and decisioning	20%	Realistic service boundaries, deterministic cases, explicit pass/manual/fail outcomes.
Code quality and tests	20%	Idiomatic Java, clear responsibilities, meaningful tests, no over-engineering.
Production mindset	15%	Auditability, security basics, error handling, observability and supportability.
Communication	10%	Clear README, assumptions, tradeoffs and what would be done next.

Interview discussion

In the follow-up discussion, candidates should be ready to explain what they chose to build, what they deliberately left out, how they would operate the service in production, and how they would extend the design to another market.